

THE WAT FRAMEWORK

The foundation for building reliable agentic workflows.

Layer	What It Is		Format	Role
Workflows	Instruction SOPs		Markdown .md	Defines steps, tools, edge
Agent	Claude itself	Code	—	Reads workflows, calls tools, handles
Tools	Execution scripts		Python .py	Does the actual work (API calls, scraping, transforms)

Why WAT? Each step at 90% accuracy = 59% success after 5 steps. Offloading execution to deterministic scripts keeps the AI focused on orchestration.

Key rule: Workflows tell the agent what order to use tools. Tools are useless without workflows. Workflows are useless without tools.

SETUP: 5 WAYS TO RUN CLAUDE CODE

1. Terminal (CLI) — Most Power

- Open terminal → cd your-project → type claude
- Full command access, most hackable, features arrive here first
- Text-only, steeper curve
- Install: npm install -g @anthropic-ai/claude-code (or per-OS command from docs)

2. Desktop App — Easiest for Beginners

- GUI with line-by-line diff review, built-in app preview, multiple parallel sessions
- Scheduled tasks run natively here
- Mac & Windows only; less customizable than CLI

3. Web (claude.ai/code) — Remote/Mobile

- Connect GitHub repo → sessions persist even with laptop off
- No local setup; works from any device/iPad
- Research preview / experimental

4. VS Code Extension — Best Daily Driver

- Install "Claude Code" from Extensions marketplace
- Files on left, agent on right — zero context switching
- Works in Cursor and Windsurf too
- Some CLI-only features require dropping into the integrated terminal

5. VPS (e.g., Hostinger) — Always-On Server

- Cloud Code runs 24/7 next to real infrastructure
- SSH in from anywhere; pair with a Telegram bridge for mobile access
- Requires basic server knowledge; watch permissions carefully

PERMISSION MODES

Toggle at the bottom of the Claude Code panel:

Mode	What Claude Can Do
Plan	Think, read files, web search — no writes or actions
Ask before edits	Read & write files freely; asks permission for bash commands
Edit automatically	Read & write without asking; still prompts for bash
Bypass permissions	Full autonomy — reads, writes, runs bash without asking

Best practice: Start in Plan mode → review & refine → switch to Bypass to execute.

To enable Bypass: Settings → search "Claude Code" → Allow dangerously skip permissions

MODELS

/model # open model picker				
Model	Speed	Strength	Cost	Use When
Haiku	Fastest	Lightweight	Cheapest	Sub-agents, bulk token processing
Sonnet	Balanced	Daily coding	Mid	~80% of
Opus	Slowest	Heavy reasoning	Most	Workplex architecture, tricky bugs

Tip: Default to Opus for quality. Switch to Haiku for high-volume sub-agent tasks.

TOKENS & CONTEXT WINDOW

- 1 token ≈ 0.75 words (~3–4 characters)
- Context window: ~200,000 tokens (Claude's working memory / notepad)
- Everything eats tokens: system prompt, CLAUDE.md, MCP servers, conversation history, files

Context rot: As tokens fill up, output quality drops sharply. Clear or compact before hitting ~60%.

Context Commands

/context	# show current token usage breakdown
/compact	# compress conversation, keep key facts
/clear	# wipe everything, start fresh
/r	# revert to an earlier point after a bad code change

Autocompact kicks in automatically at the limit — but manual compaction at ~60% keeps quality high.

CLAUDE.MD — THE SYSTEM PROMPT

Claude reads this file before every message. Think of it as onboarding docs.

What to include:

- Tech stack & project structure
- Purpose of each component
- How you want Claude to work (frameworks, constraints, style)
- File layout so Claude stays organized

Keep it lean — every token in CLAUDE.md is consumed on every message.

```
/init      # scan codebase and auto-generate a CLAUDE.md
```

Example structure:

```
# Agent Instructions
You are [role]. Your goal is [objective].

## Tech Stack
- [tool]: [purpose]

## File Structure
- workflows/    # Markdown SOPs
- tools/        # Python execution scripts
- .tmp/         # Temporary processing files
```

BUILT-IN TOOLS (WHAT YOU'LL SEE IN LOGS)

Tool	What It Does
Read	Read a file
Write	Create/overwrite a file
Edit	Patch a specific section of a file
Bash	Run a shell command in terminal
Glob	Find files matching a pattern
Grep	Search text inside files
LS	List directory contents
WebFetch	Get content from a URL
WebSearch	Search the web

You don't need to memorize these — they appear in Claude's output as it works.

SLASH COMMANDS

```
/model      # switch model (Haiku / Sonnet / Opus)
/context     # token usage breakdown
/compact    # compress conversation history
/clear      # reset conversation
/r          # revert after a bad change
/init       # generate CLAUDE.md from codebase scan
/schedule   # create a scheduled task (desktop app)
/agents     # manage sub-agents (terminal/CLI only)
```

PROMPTING BEST PRACTICES

Bad vs Good Prompts

Bad	Good
"Build me a website for my dog walking business"	"Create a landing page for Happy Paws dog walking. Hero with headline, 3 services with prices, contact form. Clean modern style, blue/white color scheme."
"I need a lead scraper for LinkedIn"	"Scrape 75 LinkedIn profiles of CTOs at SaaS companies with <200 employees. Output: name, company, LinkedIn URL, city. Stop at exactly 75."

Power Prompting Tactics

- Use Plan mode first — let Claude ask all the hard questions before building
- Tag files directly — type @filename to reference a specific asset
- Voice to text — speak faster than you type; unlocks more natural thought
- Set a finish line — "I need exactly X results. Once you have them, stop."
- Use 95% confidence rule — "Be 95% sure before moving on, otherwise ask me"
- Add brand assets — drag in logo + brand guidelines so outputs stay on-brand

MCP SERVERS

Model Context Protocol = a universal plugin port for Claude.

Instead of wiring up individual API endpoints, connect once to a service's MCP server and Claude figures out which endpoints and parameters to use.

```
# Install an MCP server (example: Firecrawl)
# Copy the install command from the service's docs, paste into CLI
"Help me install the Firecrawl MCP server using this command: [p
```

Common MCP servers: Gmail, Google Sheets, Calendar, Firecrawl, Trigger.dev, ClickUp

MCP vs Skills:

- MCP = get data & take actions (Gmail, scraping, databases)
- Skills = knowledge & custom instructions (how to design PDFs, front-end patterns)

SKILLS

Dynamic instruction sets Claude loads only when relevant — saves tokens vs. always-on system prompts.

```
# See what skills are loaded
"What skills do you have in this project?"

# Install a skill
/skill install [skill-name]    # or paste install command from sk

# Invoke a skill in a prompt
"Use the front-end design skill to build this landing page..."
```

Create your own: When you catch yourself repeating the same instructions, turn them into a skill.

- Global skills: available across all projects
- Local skills: stored in .claude/skills/ within a project

Useful built-in skills: loop, schedule, front-end design

BUILDING WORKFLOWS — STEP BY STEP

1. Drop CLAUDE.md into project root
2. Switch to Plan mode
3. Describe goal in plain language (brain dump is fine)
4. Claude asks clarifying questions → answer them
5. Review the plan; push back on anything you don't like
6. Tag brand assets / config files with @filename
7. Auto-accept plan → Claude switches to Bypass and builds
8. Watch the to-do list tick down
9. Add API keys to .env when prompted
10. Run workflow → iterate in natural language until solid
11. Deploy (Modal / Trigger.dev / scheduled task)

WAT folder structure Claude creates:

```
.tmp/        # temporary processing files (regenerated as needed)
tools/       # Python scripts (.py)
workflows/   # Markdown SOPs (.md)
.env         # API keys (never commit this)
```

DEPLOYMENT OPTIONS

Modal — Python Workflows

- Pay-per-execution (not per minute)
- \$5 free on signup, \$30 with card added
- Tell Claude: "Push this workflow to Modal, schedule it every Monday at 6am"
- Claude handles packaging, cron setup, secrets storage

Trigger.dev — TypeScript Agents

- Scheduled runs, automatic retries, queuing, orchestration
- Connect GitHub repo → auto-deploys on every push to main
- Dev environment for testing, Production for live
- Add env vars in Trigger.dev dashboard (not just in local .env)
- Install Trigger.dev MCP server for Claude to manage runs directly

```
# Tell Claude to deploy
"Push the YouTube analytics workflow to Trigger.dev,
run every Monday at 6am. Here are my project credentials:"
```

Important: When you deploy, you deploy the code (workflows + tools), not the agent itself. The self-healing ability lives in Claude Code locally — deployed code behaves deterministically.

SCHEDULED TASKS & LOOPS

Desktop App Scheduled Tasks

- Native GUI: Schedule tab → New Task
- Or type /schedule in any session
- Set name, prompt, model, mode, folder, recurrence
- Limitation: Computer must be on with desktop app open
- Missed tasks are caught up within 7 days on next launch
- Tasks are stateless — each run is a fresh session

```
# Self-improving task pattern:
Prompt: "Before running, read last-run.md. Do [task].
After finishing, overwrite last-run.md with status,
issues found, and anything the next agent should know."
```

Loop Skill (All Surfaces)

Recurring prompts within a single session (up to 3 days):

```
/loop 5m check my ClickUp for new tasks
/loop 1h remind me to stand up and stretch
/loop every 20 minutes run the review PR skill

# One-time reminder
"At 3pm remind me I have a client call"
"In 45 minutes remind me to take out the garbage"
```

Internals — 3 cron tools:

```
cron_create # schedule a recurring prompt
cron_list # see all active crons in this session
cron_delete # cancel a cron by job ID
```

Loop vs Scheduled Tasks:

	Loop	Scheduled Task
Duration	Up to 3 days	Indefinite
Context	Same session (shared)	New session each run (stateless)
Available in	All surfaces	Desktop app only (for now)
Use for	Active monitoring, reminders	Long-running automations

HOOKS

Shell commands that fire on Claude Code events (configured in settings):

```
# Example: play a sound when Claude finishes
"Set up a hook to play a sound every time you finish talking to me"

# Useful hooks
- on_session_end: play notification sound
- on_session_end: post a ClickUp message saying task is done
```

SECURITY CHECKLIST BEFORE DEPLOYING

- All secrets in .env — never hardcoded in tools or workflows
- .env listed in .gitignore
- No API keys passed directly in Claude Code chat
- Webhooks have authentication/validation
- Review permissions: what can the deployed code actually do?
- Run "Do a security review of the code before we deploy"

CONTEXT MANAGEMENT — ADVANCED

Signs of context rot:

- Claude starts making things up or repeating itself
- Outputs get generic or vague
- Errors appear that weren't there before

Fix:

```
/compact # summarize & continue (keeps history)
/clear # nuclear reset
```

Minimize tokens by:

- Keeping CLAUDE.md lean (only what's truly needed)
- Using skills instead of long system prompt sections
- Clearing between distinct tasks
- Avoiding massive file dumps — tag specific files with @

MONETIZATION MINDSET

Don't pitch: "I build agentic workflows in Claude Code"
Do pitch: "I can save you X hours/month" or "reduce process errors by Y%"

Pricing:

- Early on: hourly is fine to build trust
- Once ROI is clear: value-based pricing
- Example: system saves client \$10K/month → charge \$5K to build → 2-week payback

The doctor analogy: Don't be a pharmacist (just fill prescriptions). Be the doctor — diagnose the real constraint, then prescribe the right solution.

Engagement ladder: \$3K build → ongoing optimization → trusted partner → \$50K/year relationship

Track metrics proactively — show the client the value; don't wait for them to notice.

QUICK REFERENCE CARD

```
# Start a project
1. Open folder in VS Code
2. Drag in CLAUDE.md
3. Open Claude Code panel
4. "Initialize this project based on the CLAUDE.md file"

# Build a workflow
5. Switch to Plan mode
6. Describe goal + ask Claude to ask questions
7. Review plan, tag assets with @
8. Auto-accept → Bypass permissions → let it build

# Manage context
/context → check token usage
/compact → compress (keep going)
/clear → fresh start

# Deploy
```

COMMON MISTAKES	
Mistake	Fix
Vague goal ("build me a scraper")	Define exact inputs, outputs, and stop conditions
No plan mode before building	Always plan first; let Claude ask questions
Giving API keys in chat	Always use .env file
Not tagging brand assets	Use @filename to reference logo, guidelines
Expecting deployed code to self-heal	Self-healing only works with the local agent present
Ignoring context percentage	Compact at ~60% to avoid context
Pricing by the hour for high-ROI work	Switch to value-based pricing once ROI is provable

- BROWSER AUTOMATION WITH PLAYWRIGHT CLI
- Install via plan mode: "I want to use Playwright CLI for browser automation — research and install it"
 - Preferred over Chrome DevTools MCP (which is token-heavy — loads all tool defs into context)
 - Run /init after install to create a CLAUDE.md for the new project

Mode	When to Use
Headless (default)	Production runs — browser invisible
Headed	Debugging — opens a visible browser window

- QA Testing Loop
- Prompt: "use browser automation to test it, find bugs, fix them, retest — don't stop until passing"
 - Claude writes test → runs it → reads screenshots → patches code → reruns automatically
 - Lock it in: "do not move to the next step until this passes"

- Web Scraping
- If Google blocks automation, Claude auto-switches to DuckDuckGo
 - Use "don't stop until you find X results" for aggressive scraping

- Logged-in Sessions
- Option A: persistent Chrome profile — Claude launches browser using existing user data (already logged in)
 - Option B: headed mode handoff — you log in manually, Claude saves session cookies for future runs

- Turning Scripts into Skills
- Once reliable: "turn this into a skill so I can invoke it by name"

VERCEL DEPLOYMENT WORKFLOW

The Pipeline

Claude Code (local) → GitHub repo → Vercel (live URL, auto-deployments)

- Step-by-Step
- Build and test on localhost in Claude Code
 - Create a GitHub repo (ask Claude or do it at github.com)
 - Tell Claude: "push this to GitHub repo [URL]" — it handles git init, commit, push
 - vercel.com → Add New Project → Import GitHub repo → Deploy
 - Every future change: tell Claude to push → Vercel auto-updates (seconds)

- Key Rules
- Add secrets in Vercel dashboard (Settings → Environment Variables) — never hardcode in repo
 - Add to CLAUDE.md: "Always test on localhost first. Do not push to GitHub unless I explicitly say to."
 - Before pushing: "do a security review — check for exposed credentials"

- Custom Domain
- Default: your-project.vercel.app
 - Add custom: Vercel project → Domains → Add → update DNS A records at registrar

PIXEL AGENTS MONITORING TOOL

A VS Code extension that turns Claude Code agents into pixel art characters in a virtual office.

- Reads Claude Code's activity log; each terminal = one character, sub-agents spawn extras
- Windows only (current version)

Install

VS Code → Extensions → search "pixel agents" → install "Pixel Art" → View Pixel Agents panel → drag to sidebar → click "+ Agent"

- Folder name must NOT contain spaces or periods
- Limitations
- Shows THAT agents are working, not WHAT they're building
 - Fully local — no outbound data

MCP SERVER SETUP WALKTHROUGHS

Firecrawl (Web Scraping)

firecrawl.dev → docs → MCP Server → copy install command → Tell Claude: "connect Firecrawl MCP using this command, but cre → Reload VS Code: Ctrl+Shift+P → "Developer: Reload Window"

Tool	Use
scrape	Single page content
map	All URLs from a site
crawl	Explore all pages
extract	Structured data
search	Web search then scrape

Free tier: 500 credits, 2 concurrent requests.

- N8N MCP
- Provide Claude your N8N cloud URL + API key + skills repo URL
 - Capabilities: view/create/edit/publish workflows, inspect nodes
 - Warning: MCP JSON config stores API keys in plaintext — keep it local, don't commit

- GitHub MCP
- GitHub → Settings → Developer Settings → Fine-grained tokens → create (never-expiring, all repo perms)
 - Give token to Claude → it can create repos, push, manage commits via natural language

- Google Workspace CLI (GWS)
- Open-source CLI (not an MCP), uses bash commands
 - Give Claude the GWS GitHub repo URL: "install this and set up authentication for me"
 - OAuth setup: Google Cloud Console → new project → APIs & Services → OAuth → Desktop App → download JSON → ~/.config/gws/ → gws auth login
 - Enable individual APIs in Cloud Console (Drive, Gmail, Docs, Sheets, Slides, Calendar)
 - ~100 built-in recipe workflows; pair with headed Chrome for visual validation

BUSINESS & AGENCY BUILDING FRAMEWORK

Core Rule: Don't sell AI agents. Sell outcomes tied to time, money, or focus.

Framework: Diagnose → Solve → Value → Price

- Diagnose — Where is the business leaking time, money, or focus?
- Solve — Build a POC first, not a full production system
- Value — Hours saved x hourly rate x 52 weeks = annual value

Pricing Math

```
10 hrs/week × $25/hr = $1,000/month = $12,000/year
Automate 60% → $600/month saved → $7,200/year saved
Charge $3,000 → pays for itself in 5 months → easy yes
```

Three Client Acquisition Methods (no audience needed)

Method	How
Cold Outreach	100+ messages/day; lead with outcome not tech; offer free work or money-back guarantee
Referrals	1-2 months post-launch: "Do you know any other business owners who need this?" — 91% of happy clients refer if asked
Trojan Horse	Partner with agencies; offer free AI discovery calls to their existing clients; give 20% revenue share

Mistakes to Avoid

- Don't build before you sell — validate demand first
- Don't lead with tech jargon — lead with business outcomes
- Don't scope vaguely — document exactly what "done" means

Early-Stage Principle

Optimize for reps, not money: free/cheap work → case studies → testimonials → confidence to charge more.